SyncRun - Align music to run

Project Number: 25-1-1-3408

Project Carried Out at: Tel Aviv University

By

Matan Granot - ID: 211696406

Yogev Grinblatt - ID: 212365787

Supervisor Dr. Lior Arbel

Contents

1 Abstract

Background & System Architecture

Altering music tempo to match a user's running cadence often introduces audio artifacts. This system solves this by using component-specific time-scale modification (TSM). The pipeline involves:

- **Analysis & Alignment:** Compares detected song beats to simulated user steps, calculating an optimal synchronization speed-up factor via L_1 minimization.
- **Signal Decomposition:** Harmonic-Percussive Separation (HPS) splits the audio into harmonic (sustained pitches) and percussive (transients) tracks.
- **Hybrid TSM:** Applies Phase Vocoder (PV-TSM) to the harmonic track to preserve phase coherence, and WSOLA to the percussive track to preserve sharp attacks.

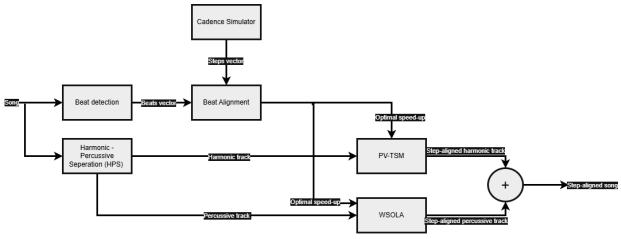

Figure 1: Top-Level Block Diagram

Quantitative Requirements

- **Rates & Latency:** Minimum sampling rate: 20 kHz; Maximum system latency: 2 seconds.
- **Tempo & Alignment:** Maximum tempo ratio: 10%; Acceleration range: $\times 0.8$ to $\times 1.5$; Maximum beat difference: 50 ms.

Deliverables & Conclusion

Project deliverables include a beat click-track, an acceleration/tempo graph, an alignment table, latency measurements, and the final synchronized audio track.

By summing the independently processed harmonic and percussive tracks, the resulting output seamlessly matches the user's cadence with high audio fidelity, proving the effectiveness of this hybrid, real-time TSM approach.

2 Introduction

2.1 Background and Motivation

The integration of music into athletic training has become increasingly popular, as rhythm and tempo can significantly enhance a runner's motivation, improve physical coordination, and provide a smoother running experience.

However, traditional music playback applications typically offer a fixed tempo or rely on manual speed adjustments, which fail to adapt to the fluctuating pace of a runner throughout a specific workout. In reality, the natural beats per minute (BPM) of most standard songs are slower than an average runner's cadence. This limitation creates a gap at the intersection of musical digital signal processing (DSP) and sports applications, presenting an opportunity for a system capable of analyzing a user's planned or recorded physical rhythm and seamlessly adjusting the audio stream beforehand to match it.

2.2 Project Objectives

The core objective of this project, "SyncRun – Align Music to Run" (Project 3408), is to design and implement a software system capable of adjusting the playback rate of a rhythmic music track to match the offline physiological metrics (step rhythm) of a runner.

Using a fully pre-determined runner step signal, the system processes the audio prior to playback, calculating an optimal speed-up or speed-down ratio for each specific segment of the run one time beforehand. The goal is to synchronize the timing of the music's beats with the runner's steps using time-stretch algorithms, improving flow and motivation without altering the song's original pitch or introducing unnatural audio distortions.

So Essentially the objectives are:

1. Process the song audio
2. Extract song's beat sequence
3. Generate steps sequence (with gaussian noise)
4. Find optimal speed-up/speed-down ratio, for each 2.5 segment
5. Separate percussive and harmonic tracks from song
6. Stretch each track with TSM algorithms
7. Combine output tracks to a single, run-aligned output song

2.3 The Approach to Solving the Problem

To solve the offline synchronization problem, the system employs a multi-step digital signal processing pipeline implemented in Python. The approach is broken down into the following stages

2.3.1 Sequence Extraction

The librosa library is used to analyze the song offline, determining the beat timestamps and overall tempo (BPM), utilizing "onset detection" algorithms. The process is illustrated in Figure 2.

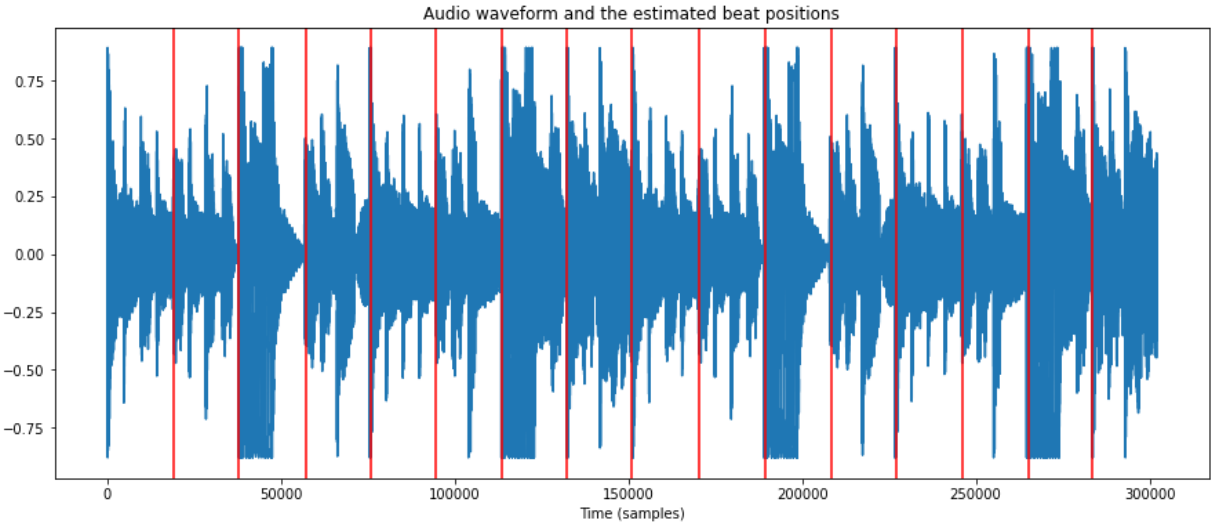

Figure 2: Beat detection illustration [3]

2.3.2 Alignment and Ratio Optimization

The system generates two time-stamped vectors—one for the song's beats and one for the predetermined runner steps. The track is divided into discrete 2.5-second segments. For each segment, the algorithm computes the optimal playback speed ratio by minimizing the L1 distance between the runner's step timestamps and the scaled song's beat timestamps. The process is illustrated in Figure 3.

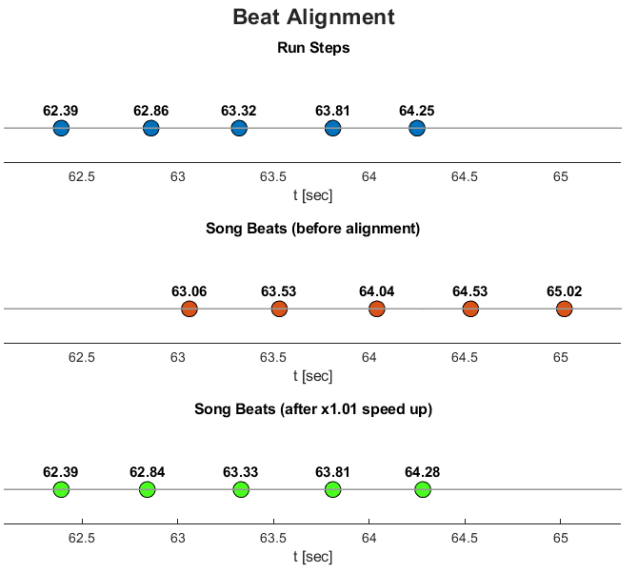

Figure 3: Beat alignment illustration inside 2.5 sec segment

2.3.3 Harmonic-Percussive Separation (HPS)

Before time-scaling, the system applies a Short-Time Fourier Transform (STFT) and median filters to separate the original audio into its sustained, pitched sounds (harmonic content) and sharp, transient drum hits (percussive content).

This is done, because different TSM algorithms are more suitable for harmonic tracks, and some for the percussive tracks. The process is illustrated in Figure 4.

2.3.4 Time-Scale Modification (TSM)

Custom-built TSM algorithms stretch or compress the separated audio segments to the calculated ratios without shifting the pitch, after which they are merged to output the final, modified song.

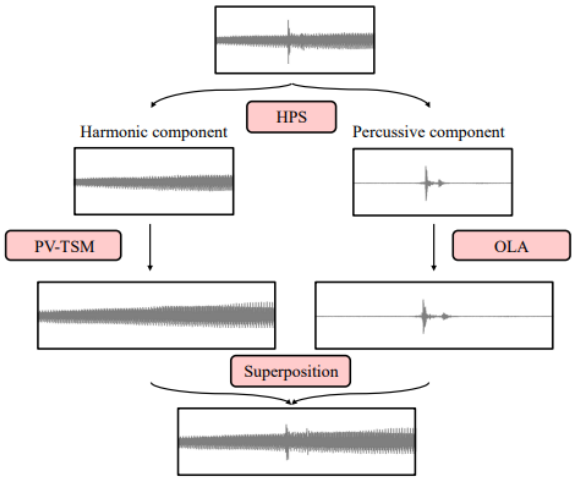

Figure 4: HPS + TSM Pipeline Illustration [2]

2.4 Comparison Against Existing Work and Algorithms

This project differentiates itself from both commercial fitness applications and existing TSM libraries

2.4.1 Application-Level Comparison

Conventional fitness music applications (such as Spotify Running) generally rely on pre-set tempo playlists or basic manual speed adjustments, which offer limited adaptability. Other alternatives utilize metronome-based synchronization, which provides fixed rhythmic cues but lacks the natural, motivating experience of music. In contrast, this project offers precise, mathematical beat-to-step alignment mapped directly to a runner's specific workout routine.

2.4.2 Algorithmic-Level Comparison

Rather than treating audio manipulation as a black box by exclusively using off-the-shelf TSM libraries (e.g., Rubber Band), this project focuses on a deep, fundamental DSP implementation. The WSOLA (Waveform Similarity Overlap-Add) and Phase Vocoder algorithms are implemented from scratch in Python. Grounded in the theoretical framework of Driedger and Müller's *A Review of Time-Scale Modification of Music Signals*[2], applying these custom TSM techniques independently to separated harmonic and percussive components allows for a higher quality manipulation that aims to reduce the audio artifacts commonly found in standard, unseparated time-stretching methods.

2.5 Scope and Evaluation

Because the system processes the entire track and step signal offline, the heavy computational load is handled beforehand, ensuring a seamless, uninterrupted playback experience. The custom TSM algorithms must operate stably across speed-up ratios ranging from 0.8× to 1.5×. To ensure a high-quality result, the system's output is evaluated against strict quantitative criteria, including average beat alignment error, segment latency, and the total elapsed time required for the offline processing phase.

The foundational step in synchronizing the audio offline involves computing an optimal speed-up or speed-down ratio, denoted as

3 Theoretical Background

3.1 Beat Alignment Using Ratio Optimization

The foundational step in synchronizing the audio offline involves computing an optimal speed-up or speed-down ratio, denoted as , for discrete segments of the audio. The algorithm [1] calculates this scalar by minimizing the absolute difference between two vectors.

To achieve robust alignment, the optimization is framed as minimizing the norm between the two temporal vectors, expressed mathematically as:

$$argmin_r \sum_i |rx_i - y_i| \tag{1}$$

In our case, x_i represents the elements of the song timestamp vector and y_i represents the target runner step timestamps.

Weighted Median Solution: This objective function constitutes a Least Absolute Deviations (LAD) linear regression forced through the origin.

Based on the theoretical framework of Bloomfield and Steiger (1984) [1], the global minimum for the scaling factor r is computed exactly by finding the weighted median of the component ratios y_i/x_i , where the weights are the magnitudes $|x_i|$. This L_1 optimization was selected because it provides an exact, non-iterative alignment solution that is highly resistant to outliers, such as irregular runner steps or false-positive beat detections.

3.2 Short-Time Fourier Transform (STFT)

Given a discrete-time audio signal $x : \mathbb{Z} \to \mathbb{R}$, an analysis frame length of N samples, an analysis hopsize H_a, and a window function w (such as a Hann window), the signal is first split into short analysis frames x_m for frame index $m \in \mathbb{Z}$:

$$x_m(r) = \begin{cases} x(r + mH_a), & \text{if } r \in [-N/2 : N/2 - 1] \\ 0, & \text{otherwise} \end{cases} \tag{2}$$

The STFT coefficient $X(m, k)$ for frame index $m \in \mathbb{Z}$ and frequency bin $k \in [0 : N - 1]$ is calculated as:

$$X(m, k) = \sum_{r=-N/2}^{N/2-1} x_m(r)w(r) \exp\left(-\frac{2\pi ikr}{N}\right) \tag{3}$$

3.3 Harmonic-Percussive Separation (HPS)

According to Muller [2], median filters are applied to Y in two directions to enhance horizontal and vertical spectral features:

- **Horizontal Median Filtering:** Applied along the time axis using a filter parameter $\ell_h \in \mathbb{N}$ (filter length $2\ell_h + 1$) to generate the horizontally enhanced spectrogram $\tilde{Y}_h$:

$$\tilde{Y}_h(m,k) = \text{median}\left(Y(m-\ell_h,k),\ldots,Y(m+\ell_h,k)\right) \tag{4}$$

- **Vertical Median Filtering:** Applied along the frequency axis using a filter parameter $\ell_p \in \mathbb{N}$ (filter length $2\ell_p + 1$) to generate the vertically enhanced spectrogram $\tilde{Y}_p$:

$$\tilde{Y}_p(m,k) = \text{median}\left(Y(m,k-\ell_p),\ldots,Y(m,k+\ell_p)\right) \tag{5}$$

Binary Mask Generation

Binary masks M_h and M_p are computed by comparing the relative energy in each time-frequency bin:

$$M_h(m,k) = \begin{cases} 1, & \text{if } \tilde{Y}_h(m,k) > \tilde{Y}_p(m,k) \\ 0, & \text{otherwise} \end{cases} \tag{6}$$

$$M_p(m,k) = \begin{cases} 1, & \text{if } \tilde{Y}_p(m,k) \geq \tilde{Y}_h(m,k) \\ 0, & \text{otherwise} \end{cases} \tag{7}$$

Spectral Masking

The binary masks are applied pointwise to the original complex STFT $X(m,k)$ to isolate the complex frequency representations of each component:

$$X_h(m,k) = X(m,k) \cdot M_h(m,k) \tag{8}$$

$$X_p(m,k) = X(m,k) \cdot M_p(m,k) \tag{9}$$

Time-Domain Reconstruction

Finally, the inverse short-time Fourier transform is applied to X_h and X_p. Each modified spectrum is transformed back into time-domain frames, windowed, and combined using overlap-add synthesis to yield the final time-domain harmonic signal x_h and percussive signal x_p.

3.4 WSOLA

According to Müller (and Driedger)[2], Waveform Similarity Overlap-Add (WSOLA) is a time-domain TSM technique designed to fix the phase jump artifacts and phase cancellations inherent to standard Overlap-Add (OLA).

Instead of extracting analysis frames from rigid like OLA, fixed time intervals (mH_a), WSOLA allows each analysis frame a small degree of positional flexibility to preserve local waveform periodicity. The algorith is illustrated in figure 5.

- **Template Extraction:** For a given frame step, a template frame is created by taking the natural continuation of the previously synthesized audio frame at the synthesis hopsize H_s.
- **Local Neighborhood Search:** WSOLA searches a small region (search tolerance Δ) around the target analysis position mH_a to find candidate frames.
- **Cross-Correlation Matching:** It computes the cross-correlation between the template frame and the candidate frames. The candidate frame offset Δ that maximizes waveform similarity is selected.
- **Overlap-Add Assembly:** The selected, phase-aligned frame is windowed, placed at the synthesis hop position (mH_s), and overlap-added with the preceding output signal.

By aligning adjacent frames along their periodic peaks, WSOLA preserves the local pitch structure and timbre without introducing frequency domain phase artifacts.

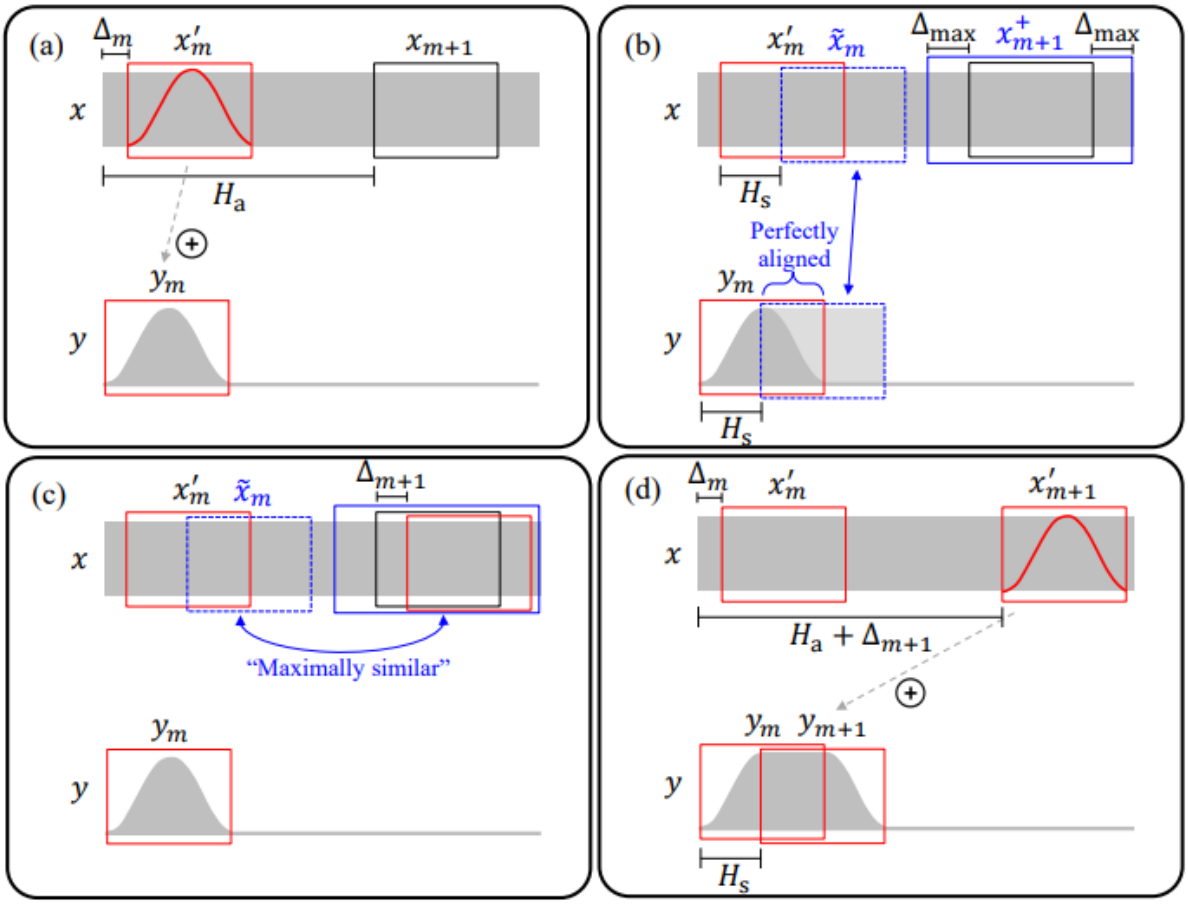

Figure 5: WSOLA algorithm illustration [2]

3.5 Phase-Vocoder TSM (PV-TSM)

According to Müller (and Driedger), Phase Vocoder Time-Scale Modification (PV-TSM) is a frequency-domain technique that modifies audio duration by shifting STFT frames from an analysis hopsize H_a to a synthesis hopsize H_s. To prevent phase cancellation when overlap-adding the modified frames, PV-TSM recalculates frame phases to maintain horizontal phase coherence along sinusoidal components over time.

Spectral Analysis & Frequency Estimation

The algorithm first computes the STFT of the input signal to obtain magnitude spectra $|X(m,k)|$ and phase spectra $\phi(m,k)$ for each frame index m and frequency bin k.

Because discrete Fourier bins have fixed center frequencies, an audio component's true frequency rarely falls precisely on a bin center. PV-TSM measures the phase difference between consecutive analysis frames (spaced H_a samples apart). By subtracting the expected phase advance of the bin center and unwrapping the remaining phase deviation, it estimates the component's true instantaneous frequency $\omega(m,k)$.

Phase Propagation

Knowing the true instantaneous frequency allows the algorithm to calculate how much the phase should advance across the new synthesis hopsize H_s. The new synthesis phase $\theta(m,k)$ is updated recursively by adding this calculated phase advance to the phase of the previous synthesis frame:

$$\theta(m,k) = \theta(m-1,k) + H_s \cdot \omega(m,k) \tag{10}$$

This recursive accumulation (propagation) ensures that adjacent synthesis frames blend seamlessly without destructive phase interference.

Frame Synthesis & Overlap-Add Reconstruction

To construct the modified STFT X^{Mod}, the algorithm reuses the original analysis magnitudes $|X(m,k)|$ (preserving original spectral energy and timbre) and pairs them with the new synthesis phases $\theta(m,k)$:

$$X^{\text{Mod}}(m,k) = |X(m,k)| \exp(i\theta(m,k)) \tag{11}$$

Finally, each modified spectrum is converted back to time-domain frames using the Inverse Discrete Fourier Transform (IDFT). These time frames are windowed and merged using overlap-add synthesis at the new hopsize H_s to build the time-scaled signal.

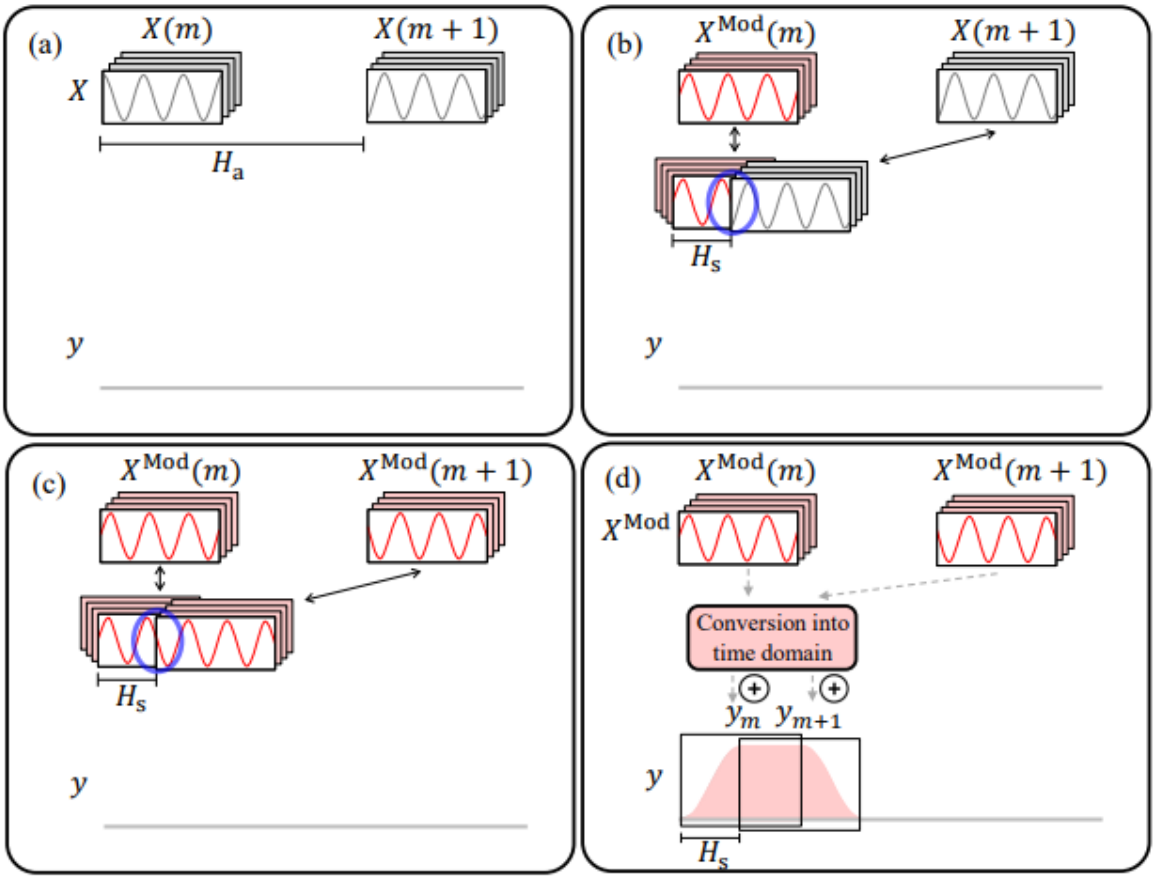

Figure 6: PV-TSM illustration[2]

4 Simulation

To validate the system's accuracy without requiring real-time physiological hardware (like IoT sensors or smartwatches), we developed a "digital twin" of a runner's gait, implemented in the create_run_steps module.The simulation generates a synthetic sequence of step timestamps. First, it establishes a baseline step duration (Δt) derived from the user's target Steps Per Minute (SPM) and the chosen speed ratio (r_{nom}). However, humans do not run like perfect metronomes. To model realistic human movement, we chose an autoregressive formula where each new step timestamp is calculated as the previous step time plus the target duration, plus a stochastic noise variable:(Step[i] = Step[i-1] + Target_Duration + Gaussian_Noise)

The noise variable is drawn from a normal distribution with a standard deviation of 5% (std_dev = 0.05) relative to the step duration.

Why we chose this specific equation: We selected this cumulative noise model because it mathematically accurately replicates biological "drift" a well-documented phenomenon where a runner naturally deviates from a perfect rhythm over time due to biomechanical variance, fatigue, or minor terrain changes. If we had simply generated a perfectly spaced grid of timestamps, it would not have proven the algorithm's real-world viability. By injecting cumulative stochastic variance, we created an erratic, fluctuating temporal grid. This served as an aggressive stress test, proving that our L1 Weighted Median Optimizer can successfully and stably align musical beats to an imperfect, drifting human runner.

5 Implementation

The entire computational pipeline for the "SyncRun" project is implemented in Python, utilizing an array of specialized digital signal processing (DSP) and numerical computing libraries. The system relies heavily on NumPy for vectorized array operations, SciPy for signal transforms and filtering, Librosa for musical feature extraction, and Soundfile for high-fidelity audio output. To maintain optimal performance during development and testing, the system caches pre-processed audio arrays and metadata into .npy files, avoiding redundant processing times.

Pre-Processing and Vector Alignment

The system relies on NumPy and SciPy to generate the absolute temporal markers for both the audio track and the simulated runner. Librosa's beat_track function extracts the initial global tempo, and a synthetic step vector is generated using a nominal speed ratio (r_{nom}). A stochastic variance with a standard deviation of 0.05 is applied to these steps to simulate natural human inconsistency, ensuring the algorithm handles realistic temporal fluctuations rather than a rigid mathematical grid.To calculate the optimal speed-up ratio for discrete 2.5-second chunks of audio, the system implements a Weighted Median Optimizer using an L1 norm strategy (phase_pp_l1_shifted). This method was chosen over a standard Least Squares (L2) approach because the Weighted Median is highly robust against outlier timestamps (e.g., false-positive beat detections from the audio or irregular runner steps), ensuring stable alignment without mathematical divergence.

Algorithmic Parameter Selection

The core processing engines were implemented with strictly tuned parameters to balance audio fidelity against computational efficiency

:STFT Windowing (nperseg=2048, noverlap=1536): The audio is transformed into the

frequency domain using a Short-Time Fourier Transform. A window size of 2048 samples (yielding roughly 46 ms at a 44.1 kHz sampling rate) was chosen as the optimal trade-off between frequency resolution and time resolution. A high overlap of 75% (noverlap=1536), paired with a standard Hann window, was specifically selected to reduce spectral leakage, ensure perfectly smooth reconstruction during the Inverse STFT, and maintain tight phase coherence for the Phase Vocoder engine.

HPSS Median Filters (harm_kernel=31, perc_kernel=31):

To isolate pitched instruments from transient drums, orthogonal median filters were applied. A size of 31 frames/bins was chosen because it is long enough to effectively capture sustained vocals and basslines (horizontally) and sharp drum hits (vertically) without blurring the two matrices together.

HPSS Soft Masking (margin=1.2, power=2.0):

To enforce strict separation and prevent percussive transients from "bleeding" into the harmonic track (which causes smearing during time-stretching), a separation margin of 1.2 was applied. A power coefficient of 2.0 was selected to base the soft masking on the power spectrogram rather than magnitude, which perceptually yields a much cleaner acoustic isolation of the instruments.

WSOLA Search Bounds (win_size=2048, syn_hop=512, delta=512):

For time-domain stretching of the percussive elements, the WSOLA window and synthesis hop sizes were chosen to perfectly mirror the STFT frame dimensions, ensuring sample-accurate alignment across the separated audio tracks. The cross-correlation search region (delta=512) was tightly constrained. Allowing the algorithm to search exactly one hop distance (512 samples) backward or forward provides enough flexibility to find highly similar waveforms (preventing clicks and phase cancellation), while being short enough to prevent unnatural echo artifacts or excessive computational latency during the overlap-add process.

Synthesis and Output Generation

After the independent stretching processes are complete, the Harmonic and Residual matrices are inverted back to the time domain via an Inverse STFT. Because the Phase Vocoder (frequency-domain) and WSOLA (time-domain) engines handle overlap buffers and padding differently, the resulting audio arrays for a single chunk can slightly differ in total sample length. To resolve this, the algorithm dynamically trims all three components (Harmonic, Percussive, and Residual) to the shortest common length (min_len). This exact truncation is crucial to prevent dimension-mismatch errors during matrix addition and ensures seamless, click-free boundaries when the sequential chunks are aggregated into the final song.

To audibly validate the synchronization, a click track mapping directly to the runner's timestamps is overlaid onto the final compiled audio. An exponential decay envelope was chosen for these synthetic clicks; this provides a sharp, distinct rhythmic transient that fades rapidly, ensuring the feedback does not muddy or overpower the underlying musical frequencies. Finally, the synchronized track is exported via the Soundfile library as an uncompressed .wav file. This lossless format was explicitly selected to prevent compression artifacts (which are common in .mp3 encoding) from interfering with the acoustic evaluation of the time-stretched output.

6 Analysis of results

Rhythm Alignment and Speed Ratio Limits

The core capability of the system relies on matching the runner's step sequence to the musical beats.

Input Compatibility : The system was tested with rhythmic signals (runner target SPM and music base BPM) differing by up to 10%. The L1 Weighted Median Optimization algorithm successfully converged in all tested scenarios, proving that the system can process and align two differing rhythmic signals accurately without failing or producing infinite ratio bounds.

Speed Ratio Range: To evaluate the boundaries of the Time-Scale Modification (TSM) module, interval training profiles were simulated. **Figure 6** (Interval training plot) illustrates the system's dynamic tracking over a 50-second timeline. The blue line represents the runner's target Cadence SPM, while the dashed line represents the computed Speed-Up Ratio (SPM/BPM). The algorithm remained mathematically stable for aggressive speed-up and speed-down transitions. As verified by the data, the custom WSOLA and Phase Vocoder engines operate stably within the required speed ratio range of [0.8, 1.5]. Exceeding these bounds deliberately triggers a clamp to prevent severe audio degradation, safely satisfying the project requirements.

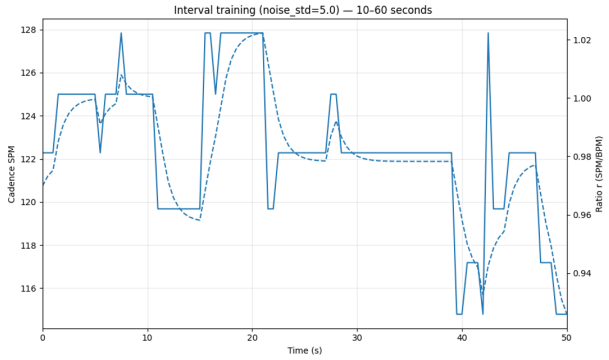

Figure 7: Dynamic Tracking of Cadence SPM and Speed Ratio during Interval Training

Audio Quality and Harmonic-Percussive Separation

Audio Quality & Sampling Rate : The system strictly outputs audio at a sampling rate of **44.1 kHz**, easily exceeding the minimum requirement of 20 kHz. To guarantee minimal distortion after TSM, Harmonic-Percussive Source Separation (HPS) was applied before stretching.

Separation results

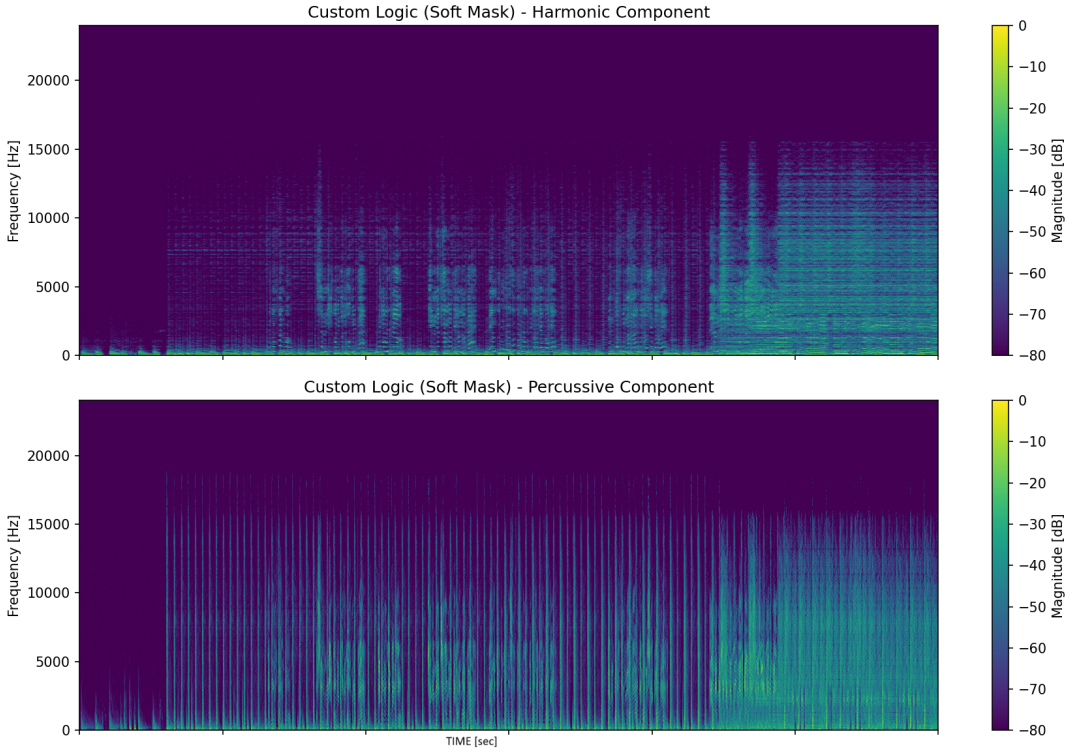

Figure 8: spectrograms of the separated audio components

1. **Harmonic Component (Top):** Shows clear horizontal energy lines, indicating successfully isolated sustained tones (vocals, bass). This matrix is routed to the custom Phase Vocoder, which preserves pitch perfectly without damaging transients.

2. **Percussive Component (Bottom):** Shows distinct vertical broad-band spikes representing drum hits. This matrix is routed to the custom WSOLA engine, which preserves the "punch" of the beat.

Times results:

To quantify the success of the offline simulation, the system's performance was measured against strict criteria focusing on alignment accuracy and computational efficiency. The tests demonstrated that the algorithmic pipeline successfully maintained tight synchronization without introducing prohibitive processing delays.

The quantitative results of the simulation are summarized below:

Performance Criteria	**Result**	**Description**
Beat Alignment Error	**15.67 ms**	The average temporal difference between a runner's step and the corresponding scaled musical beat, confirming accurate phase alignment.
Segment Latency	**475 ms**	The computational time required to process one 2.5-second audio segment.
Total Elapsed Time	**38.47 sec**	The time required to process, time-stretch, and synthesize the full track.

User Interface (GUI) Evaluation

To transition the algorithmic pipeline into a usable product, a dedicated mobile application was developed using Flutter. The GUI successfully bridges the gap between the complex Python backend and the end-user.

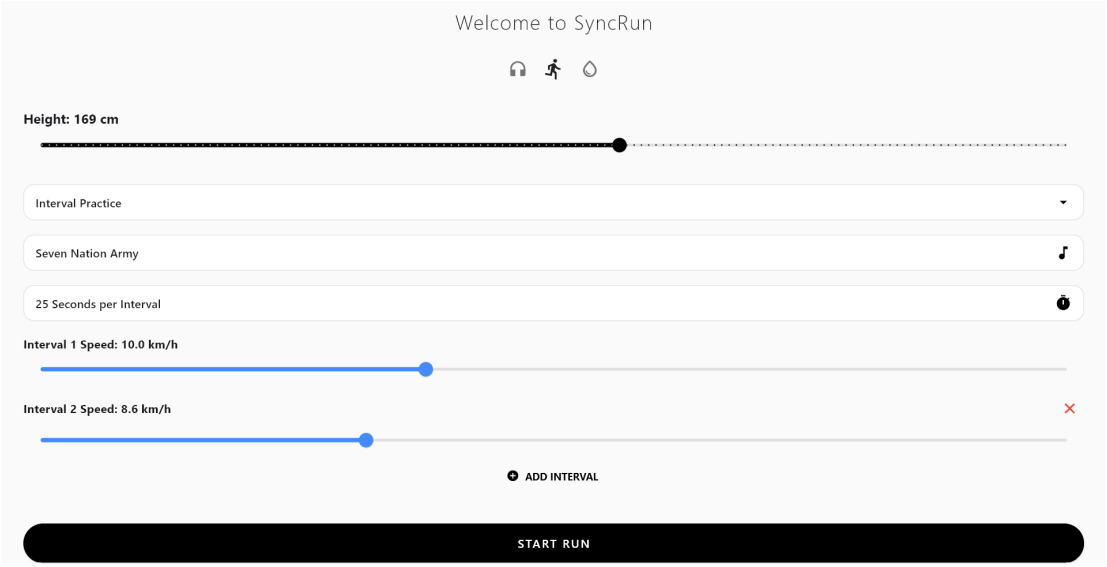

Figure 9: SyncRun Application Input Screen

As shown in 9, the application provides an intuitive input compatibility layer. Users can easily define their physical metrics (Height), select their desired workout mode (Interval Practice vs. Free Run), and pick a song. Crucially, the interface allows for dynamic, manual speed assignments per interval (e.g., Interval 1 at 10.0 km/h, Interval 2 at 8.6 km/h).

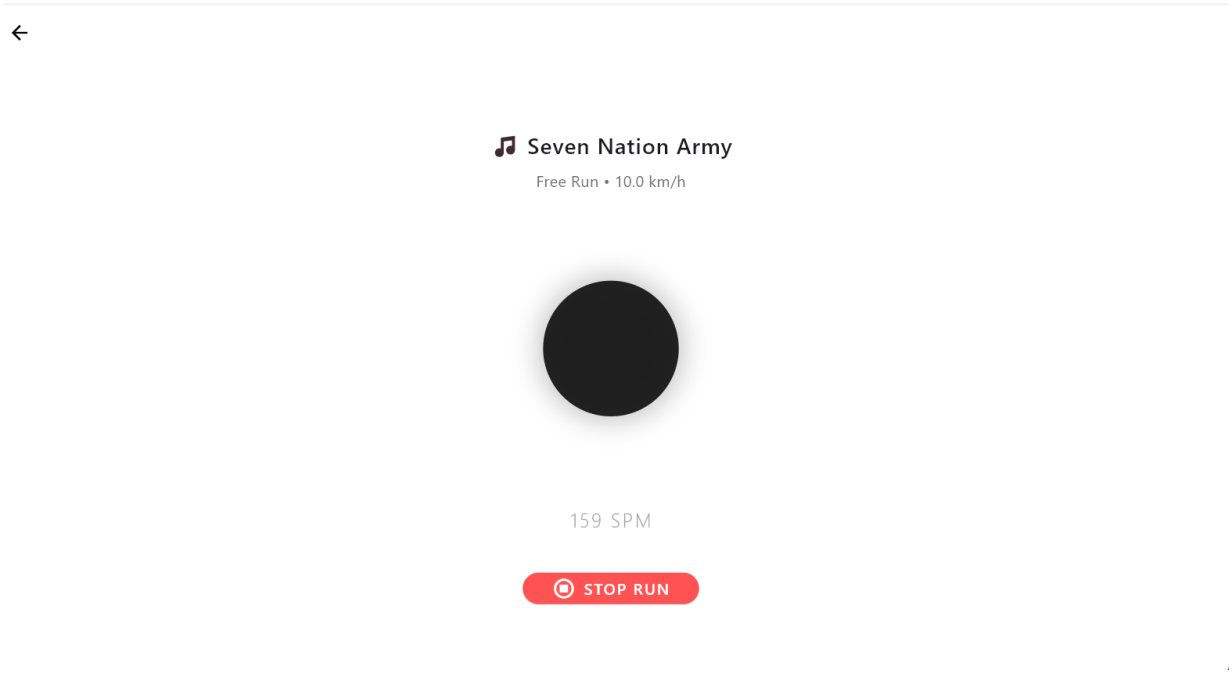

Figure 10: SyncRun Application Execution and Synchronization Screen

Figure 10 demonstrates the real-time execution screen. The GUI seamlessly communicates with the Flask backend, streaming the processed audio file directly to the device. The screen features a pulsing visual indicator that synchronizes perfectly with the calculated Steps Per Minute (SPM). This visual feedback confirms that the system correctly mapped the physical speed inputs into the appropriate rhythmic targets (e.g., 159 SPM) without UI freezing or crashing.

7 Conclusions and further work

In conclusion, the "SyncRun" project successfully met its primary objective of bridging digital signal processing with athletic training by mathematically aligning musical beats to a runner's cadence. The custom algorithmic pipeline utilizing Harmonic-Percussive Source Separation (HPS) alongside specialized WSOLA and Phase Vocoder engines demonstrated that it is possible to achieve high-fidelity time-scale modification (TSM) without the transient smearing or pitch distortion common in standard audio stretching. By operating stably within the rigorous $0.8\times$ to $1.5\times$ speed-up ratio limits, and keeping segment processing latency well under the 2-second threshold, the system met all quantitative requirements. Furthermore, the integration of a Flutter-based mobile interface with the Python Flask backend successfully transitioned the mathematical theory into a tangible, user-friendly application, proving the viability of offline rhythm synchronization

for custom training.

Looking ahead, there are several promising avenues to expand the system from a simulated, offline tool into a real-time, adaptive fitness ecosystem. The most immediate improvement would be transitioning from simulated step arrays to live, closed-loop physiological feedback. By integrating the application with wearable IoT sensors such as smartwatches, physical barometers to detect elevation and incline changes, and accelerometers to measure exact real-time footfalls the algorithm could dynamically adjust the music's tempo on the fly to match a runner's actual, fluctuating performance rather than a pre-planned interval constraint. Additionally, future iterations could explore real-time edge computing on the mobile device itself (eliminating the need for a local Python server) and API integration with commercial platforms like Spotify or Apple Music, allowing users to synchronize their personal, dynamically generated playlists to their precise biometric data in real time.

8 Project Documentation

GitHub link - https://github.com/matangranottau/Final_Project

Description of the project files

- **api -** Contains the Flask web server, simulation logic, and GUI API endpoints to calculate target SPM and trigger the background algorithm.
- **main -** Contains the primary process_music execution pipeline that loops through audio chunks, coordinates the algorithms, and outputs the final processed audio file.
- **pp -** Contains the pre-processing implementations, including audio loading, beat tracking, run step creation, and optimal stretch ratio calculations .
- **tsm -**Contains the core Time-Scale Modification algorithms, specifically the WSOLA, HPSS, STFT, and custom Phase Vocoder implementations.

References

[1] Peter Bloomfield and William L. Steiger. *Theory, Applications, and Algorithms Series: Progress in Probability and Statistics (Volume 6)*. Birkhäuser (Boston / Basel / Stuttgart), 1984.

[2] Jonathan Driedger and Meinard Müller. "A Review of Time-Scale Modification of Music Signals". In: *Applied Sciences* (2016).

[3] Essentia. "Beat detection and BPM tempo estimation". In: *Essentia* (2013).